Week 1: I. Given an array of integers, write an algorithm and a program to left rotate the array by specific number of elements. Input format: The first line contains number of test cases, T. For each test case, there will be three input lines. First line contains n (the size of array). Second line contains n space-separated integers describing array. Third line will take number (k<=n) of times elements of array is rotated. Output format: The output will have T number of lines. For each test case, output will be the new rotated array

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 1
*/
#include <stdio.h>
int main() {
   int t;
   printf("Enter number of test cases: ");
   scanf("%d", &t);
for (int x = 1; x <= t; x++) {
       int n, k;
       printf("\nTest case %d:\n", x);
      printf("Enter size of array: ");
       scanf("%d", &n);
      int a[n];
       printf("Enter %d elements: ", n);
       for (int i = 0; i < n; i++) {
          scanf("%d", &a[i]);
       }printf("Enter number of rotations (k <= %d): ", n);
       scanf("%d", &k);
  printf("Rotated array: ");
       for (int i = 0; i < n; i++) {
          printf("%d ", a[(i + k) % n]);
       }   printf("\n");
   }  return 0;}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc wk1q1.c -o
wk1q1 } ; if ($?) { .\wk1q1 } Enter the size of an array:5
 Enter the elements of an array:1 2 3 4 5
 Enter the value by which you want to left rotate your array:2
 Final array:3 4 5 1 2
 Enter the size of an array:6
Enter the elements of an array:2 4 6 7 4 4
 Enter the value by which you want to left rotate your array:4
 Final array:4 4 2 4 6 7
Enter the size of an array:5
 Enter the elements of an array:34 45 23 54 66
 Enter the value by which you want to left rotate your array:1
 Final array:45 23 54 66 34
```

II. Given an unsorted array of integers and two numbers a and b. Design an algorithm and write a program to find minimum distance between a and b in this array. Minimum distance between any two numbers a and b present in array is the minimum difference between their indices. Input format: The first line contains number of test cases, T. For each test case, there will be three input lines. First line contains n (the size of array). Second line contains n space-separated integers describing array. Third line will take two numbers a and b. Output format: The output will have T number of lines. For each test case, output will be the minimum distance between a and b.

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 2
*/

#include <stdio.h>
#include <stdlib.h>  // for abs()
int main() {
   int t;
   printf("Enter number of test cases: ");
   scanf("%d", &t);
 for (int x = 1; x <= t; x++) {
       int n, a, b;
       printf("\nTest case %d:\n", x);
 printf("Enter size of array: ");
       scanf("%d", &n);
 int arr[n];
       printf("Enter %d elements: ", n);
       for (int i = 0; i < n; i++) {
          scanf("%d", &arr[i]);
       }  printf("Enter two numbers a and b: ");
       scanf("%d %d", &a, &b);
 int posA = -1, posB = -1;
       int minDist = n;

       for (int i = 0; i < n; i++) {
          if (arr[i] == a) posA = i;
          if (arr[i] == b) posB = i;
         if (posA != -1 && posB != -1) {
             int dist = abs(posA - posB);
             if (dist < minDist) {
                minDist = dist;
             }
          }
       }  printf("Minimum distance between %d and %d is: %d\n", a, b, minDist);
   }return 0;}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc wk1q2.c -o
wk1q2 } ; if ($?) { .\wk1q2 } enter the size of the array: 10
 enter the elemnts of the array:1 2 3 4 5 6 2 4 5 2
enter the values of a and b:2 4
 the min dis is 1 enter the size of the array: 8
 enter the elemnts of the array:1 2 3 4 5 5 6 7
 enter the values of a and b:5 5
 the min dis is 0 enter the size of the array: 5
 enter the elemnts of the array:1 2 3 4 5
 enter the values of a and b:1 7
one of the element is not present in the array!!
```

III. Given an array of nonnegative integers, where all numbers occur even number of times except one number which occurs odd number of times. Write an algorithm and a program to find this number. (Time complexity = O(n)) Input format: The first line contains number of test cases, T. For each test case, there will be two input lines. First line contains n (the size of array). Second line contains space-separated integers describing array. Output format: The output will have T number of lines. For each test case, output will be the element which occured odd number of times. If no such element is present in the array, then print “No such element present”.

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 3
*/

#include <stdio.h>

int main() {
   int t;
   printf("Enter number of test cases: ");
   scanf("%d", &t);

   for (int x = 1; x <= t; x++) {
      int n;
      printf("\nTest case %d:\n", x);

      printf("Enter size of array: ");
      scanf("%d", &n);

      int arr[n], res = 0;
      printf("Enter %d elements: ", n);
      for (int i = 0; i < n; i++) {
         scanf("%d", &arr[i]);
         res ^= arr[i];   // XOR operation
      }

      if (res != 0)
         printf("Element occurring odd number of times: %d\n", res);
      else
         printf("No such element present\n");
   }

   return 0;
}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc wk1q3.c -o
wk1q3 } ; if ($?) { .\wk1q3
Enter the size of an array:5
Enter an array of non negative integers:1 2 2 3 3
The number which occurs odd number of times is: 1
Enter the size of an array:7
Enter an array of positive integers:1 1 5 5 5 4 4
The number which occurs odd number of times is: 5
```

Week 2: I. Given a matrix of size n X n, where every row and every column is sorted in increasing order. Write an algorithm and a program to find whether the given key element is present in the matrix or not. (Linear time complexity) Input Format: First line contains n (the size of matrix). For next n input lines, each line contains space-separated n integers describing each row of the matrix. Last line of input will contain key integer to be searched Output Format: Output will be “Present” if the key element is found in the array, otherwise print “Not Present”

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - WEEK 2 : 1
*/

#include <stdio.h>
int main() {
   int n, key;
   printf("Enter size of matrix: ");
   scanf("%d", &n); int mat[n][n];
   printf("Enter elements of matrix (%d x %d):\n", n, n);
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         scanf("%d", &mat[i][j]);
      }   } printf("Enter key to search: ");
   scanf("%d", &key);
 int i = 0, j = n - 1, found = 0;
   while (i < n && j >= 0) {
      if (mat[i][j] == key) {
         found = 1;
         break;
      } else if (mat[i][j] > key) {
         j--;
      } else {
         i++;   }   }   if (found)
  printf("Present\n");
   else   printf("Not Present\n");   return 0;}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc wk2q1.c -o
wk2q1 } ; if ($?) { .\wk2q1
enter the size of the 2d arr (n*n):
3
enter the elemnts of 2d array:
1 2 3
5 6 7
4 9 10
enter the key:3
present!!
enter the size of the 2d arr (n*n):
3
enter the elemnts of 2d array:
1 2 3
4 5 6
7 8 9
enter the key:3
present!!
enter the size of the 2d arr (n*n):
3
enter the elemnts of 2d array:
1 2 3
4 5 6
4 5 6
enter the key:5
present!
```

II. Given a boolean matrix (contains only 0 and 1) of size m X n where each row is sorted, write an algorithm and a program to find which row has maximum number of 1's. (Linear time complexity) Input Format: First line contains m and n (the size of matrix). For next m input lines, each line contains space-separated n integers describing each row of the matrix. Output Format: Output will be row number which has maximum number of 1's. If all the elements of matrix is 0 then print “Not Present”.

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - WEEK 2 : 2
*/

#include <stdio.h>

int main() {
    int m, n;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    int mat[m][n];
    printf("Enter elements of matrix (%d x %d) [only 0 and 1]:\n", m, n);
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &mat[i][j]);
        }
    }

    int row = -1, j = n - 1;
    for (int i = 0; i < m; i++) {
        while (j >= 0 && mat[i][j] == 1) {
            row = i;
            j--;
        }
    }

    if (row == -1)
        printf("Not Present\n");
    else
        printf("Row with maximum 1's is: %d\n", row + 1);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc que8.c -o que8 } ;
if ($?) { .\que8 }
enetr m,n:3 3
enter the values in the array (1 or 0's):
1 1 1
0 0 0
1 0 1
row contaning max 1's: 0
enetr m,n:3 3
enter the values in the array (1 or 0's):
1 1 1
1 1 1
0 0 0
row contaning max 1's: 0
```

III.Given a matrix of size n X n containing positive integers, write an algorithm and a program to rotate the elements of matrix in clockwise direction. Input Format: First line contains n (the size of matrix). For next n input lines, each line contains space-separated n integers describing each row of the matrix. Output Format: Output will be rotated matrix.

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - WEEK 2 : 3
*/

#include <stdio.h>

int main() {
    int n;
    printf("Enter size of matrix: ");
    scanf("%d", &n);

    int mat[n][n];
    printf("Enter elements of matrix (%d x %d):\n", n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &mat[i][j]);
        }
    }

    // Step 1: Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = mat[i][j];
            mat[i][j] = mat[j][i];
            mat[j][i] = temp;
        }
    }

    // Step 2: Reverse each row
    for (int i = 0; i < n; i++) {
        for (int j = 0, k = n - 1; j < k; j++, k--) {
            int temp = mat[i][j];
            mat[i][j] = mat[i][k];
            mat[i][k] = temp;
        }
    } printf("Rotated matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }   return 0;}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc wk2q3.c -o
wk2q3 } ; if ($?) { .\wk2q3 }
Enter size of matrix: 3
Enter elements of matrix (3 x 3):
1 2 3
4 5 6
7 8 9
Rotated matrix:
7 4 1
8 5 2
9 6 3
Enter size of matrix: 4
Enter elements of matrix (4 x 4):
1 2 3 4
1 5 6 7
6 8 7 5
7 8 5 7
Rotated matrix:
7 6 1 1
8 8 5 2
5 7 6 3
7 5 7 4
```

Week 3: I. Design an algorithm and a program to implement stack using array. The program should implement following stack operations: a) Create() - create an empty stack b) Push(k) - push an element k into the stack c) Pop() - pop an element from the stack snd return it d) IsEmpty() - check if stack is empty or not e) Size() - finds the size of the stack f) Print() - prints the contents of stack Input Format: Ask user first whether it wants to push/pop/find the size of the stack, then accordingly perform operation. Output Format: Output will be the final stack contents if push or pop operation is performed. Output will be size of the stack if user asks for size.

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - WEEK 3 : 1
*/

#include <stdio.h>
#define MAX 100

int stack[MAX], top = -1;

void push(int k) {
   if (top == MAX - 1) {
      printf("Stack Overflow\n");
   } else {
      stack[++top] = k;
      printf("%d pushed into stack\n", k);
   }
}

void pop() {
   if (top == -1) {
      printf("Stack Underflow\n");
   } else {
      printf("Popped element: %d\n", stack[top--]);
   }
}

void isEmpty() {
   if (top == -1)
      printf("Stack is Empty\n");
   else
      printf("Stack is Not Empty\n");
}

void size() {
   printf("Size of stack: %d\n", top + 1);
}

void print() {
   if (top == -1) {
      printf("Stack is Empty\n");
   } else {
```

```
        printf("Stack contents: ");
        for (int i = 0; i <= top; i++) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main() {
    int ch, val;
    while (1) {
        printf("\nChoose operation:\n");
        printf("1. Push\n2. Pop\n3. Size\n4. Print\n5. IsEmpty\n6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &val);
                push(val);
                break;
            case 2:
                pop();
                break;
            case 3:
                size();
                break;
            case 4:
                print();
                break;
            case 5:
                isEmpty();
                break;
            case 6:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31\" ; if ($?) { gcc wk3q1.c -o
wk3q1 } ; if ($?) { .\wk3q1 }
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
1
stack has been created succesfully!!
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
2
enter the item to be pushed:4
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
2
enter the item to be pushed:5
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
2
enter the item to be pushed:6
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
3
the value to be popped 6
enter your choice-:(
1->Create()
2->Push(k)
```

```
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
4
the stack is not empty!
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
6
5 4
enter your choice-:(
1->Create()
2->Push(k)
3->Pop()
4->IsEmpty()
5->Size()
6->Print())
8
invalid choice entered!!
```

II. Given an expression string consisting of opening and closing brackets “{“,”}”,”(“,”)”,”[“,”]”, design an algorithm and a program to check whether this expression has balanced paranthesis or not. Input Format: The first line contains number of test cases, T. For each test case, there will be expression string. Output Format: The output will have T number of lines. For each test case, output will be “Balance”, if brackets are balanced otherwise print “Unbalanced”.

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - WEEK 3 : 2
*/

#include <stdio.h>
#define MAX 1000

int main() {
   int t;
   printf("Enter number of test cases: ");
   scanf("%d",&t);
   getchar(); // consume newline

   for (int tc=1; tc<=t; tc++) {
      char s[MAX+5], st[MAX+5];
      int top=-1, ok=1;

      printf("\nEnter expression for Test Case %d: ", tc);
      scanf("%s", s);

      for (int i=0; s[i]!='\0'; i++) {
         char c=s[i];
         if (c=='(' || c=='[' || c=='{') st[++top]=c;
         else if (c==')' || c==']' || c=='}') {
            if (top==-1) { ok=0; break; }
            char o=st[top--];
            if ((o=='(' && c!=')') || (o=='[' && c!=']') || (o=='{' && c!='}')) { ok=0; break; }
         }
      }
      if (top!=-1) ok=0;

      if (ok) printf("Balance\n");
      else    printf("Unbalanced\n");
   }
   return 0;
}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\\Desktop\KRITIKA31\" ; if ($?) { gcc que10.c -o que10
} ; if ($?) { .\que10 }
enter the string:
(){}((){{(()}}
the string is balanced!!
enter the string:
(){{))
the string is unbalanced!
```

III.Given a string of opening and closing paranthesis, design an algorithm and a program to find the length of the longest valid paranthesis substring. Valid paranthesis substring is a string which contains balanced paranthesis. Input Format: The first line contains number of test cases, T. For each test case, there will be string of paranthesis. Output Format: The output will have T number of lines. For each test case, output will be length of longest valid paranthesis substring

```
/*
NAME    - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - WEEK 3 : 3
*/

#include <stdio.h>
#include <string.h>

int main() {
    int t;
    printf("Enter number of test cases: ");
    scanf("%d", &t);

    for (int tc = 1; tc <= t; tc++) {
        char s[1000];
        printf("\nEnter parenthesis string for Test Case %d: ", tc);
        scanf("%s", s);

        int n = strlen(s);
        int stack[1000];
        int top = -1;

        // initialize stack with -1
        stack[++top] = -1;
        int maxlen = 0;

        for (int i = 0; i < n; i++) {
            if (s[i] == '(') {
                stack[++top] = i;
            } else {
                top--; // pop
                if (top == -1) {
                    stack[++top] = i; // reset base
                } else {
                    int len = i - stack[top];
                    if (len > maxlen) maxlen = len;
                }
            }
        }

        printf("Length of longest valid parenthesis substring: %d\n", maxlen);
    }
    return 0;
}
```

OUTPUT:

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31
\" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }
enter the size of array
7
enter the input
((()))(
6
enter the size of array
8
enter the input
()((()))
8
```

Week4:
I. *Given an empty stack, design an algorithm and a program to reverse a string using this stack (with and without recursion).
Input Format:
The first line contains number of test cases, T.
For each test case, there will be string of characters.
Output Format:
The output will have T number of lines. For each test case, output will be new reversed string.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 10
*/

```
#include <stdio.h>
#include <string.h>

char stack[1000];
int top = -1;

void push(char c) {
   stack[++top] = c;
}

char pop() {
   return stack[top--];
}

void reverseStack(char s[]) {
   int n = strlen(s);
   top = -1;
   for(int i = 0; i < n; i++)
      push(s[i]);
   for(int i = 0; i < n; i++)
      s[i] = pop();
}

int main() {
   int t;
   char s[1000];

   printf("Enter number of test cases: ");
   scanf("%d", &t);

   while(t--) {
      printf("Enter string: ");
      scanf("%s", s);
      reverseStack(s);
      printf("Reversed: %s\n", s);
   }
   return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of test cases: 3
Enter string: hello
Reversed: olleh
Enter string: kritika
Reversed: akitirk
Enter string: goat
Reversed: taog
```

```c
#include <stdio.h>
#include <string.h>

void reverseRec(char s[], int i, int n, char out[]) {
    if(i == n) return;
    out[n-i-1] = s[i];
    reverseRec(s, i+1, n, out);
}

int main() {
    int t;
    char s[1000], out[1000];

    printf("Enter number of test cases: ");
    scanf("%d", &t);

    while(t--) {
        printf("Enter string: ");
        scanf("%s", s);
        int n = strlen(s);
        reverseRec(s, 0, n, out);
        out[n] = '\0';
        printf("Reversed: %s\n", out);
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of test cases: 3
Enter string: hello
Reversed: olleh
Enter string: world
Reversed: dlrow
Enter string: stack
Reversed: kcats
```

II. Design an algorithm and a program to implement two stacks by using only one array i.e. both the stacks should use the same array for push and pop operation. Array should be divided in such a manner that space should be efficiently used if one stack contains very large number of elements and other one contains only few elements.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 11

```
#include <stdio.h>

int arr[100], top1 = -1, top2;

void push1(int x) {
   if (top1 + 1 == top2) printf("Stack1 Overflow\n");
   else {
      top1++;
      arr[top1] = x;
   }
}

void push2(int x) {
   if (top1 + 1 == top2) printf("Stack2 Overflow\n");
   else {
      top2--;
      arr[top2] = x;
   }
}

int pop1() {
   if (top1 == -1) {
      printf("Stack1 Underflow\n");
      return -1;
   }
   return arr[top1--];
}

int pop2() {
   if (top2 == 100) {
      printf("Stack2 Underflow\n");
      return -1;
   }
   return arr[top2++];
}

int main() {
   int n, choice, val;
   top2 = 100;

   while (1) {
```

```
        printf("1 Push Stack1\n2 Push Stack2\n3 Pop Stack1\n4 Pop Stack2\n5 Exit\n");
        scanf("%d", &choice);

        if (choice == 1) {
            scanf("%d", &val);
   push1(val);
        }
        else if (choice == 2) {
            scanf("%d", &val);
            push2(val);
        }
        else if (choice == 3) {
            printf("%d\n", pop1());
        }
        else if (choice == 4) {
            printf("%d\n", pop2());
        }
        else if (choice == 5) break;
        else printf("Invalid\n");
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }

1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
1
50
1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
1
49
1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
2
32
1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
2
43
1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
3
49
1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
4
43
1 Push Stack1
2 Push Stack2
3 Pop Stack1
4 Pop Stack2
5 Exit
5
```

III. I. Given an expression in the form of postfix representation, design an algorithm and a program to find result of this expression.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 12

```
#include <stdio.h>
#include <ctype.h>

int stack[100];
int top = -1;

void push(int x) {
   stack[++top] = x;
}

int pop() {
   return stack[top--];
}

int main() {
   char exp[100];
   int i, a, b, r;

   scanf("%s", exp);

   for (i = 0; exp[i] != '\0'; i++) {
      if (isdigit(exp[i])) {
         push(exp[i] - '0');
      } else {
         b = pop();
         a = pop();
         if (exp[i] == '+') r = a + b;
         else if (exp[i] == '-') r = a - b;
         else if (exp[i] == '*') r = a * b;
         else if (exp[i] == '/') r = a / b;
         push(r);
      }
   }

   printf("%d", pop());
   return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter postfix expression: 53+82-*
Result: 48
```

WEEK5:Design an algorithm and a program to implement queue using array. The program should implement following queue operations: a) Create() - create a queue b) EnQueue(k) - insert an element k into the queue c) DeQueue() - delete an element from the queue d) IsEmpty() - check if queue is empty or not e) Size() - finds the size of the queue.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 13

```
#include <stdio.h>

int q[100], front = -1, rear = -1, n;

void create() {
   front = -1;
   rear = -1;
}

void enqueue(int x) {
   if (rear == n-1) {
      printf("Overflow\n");
   } else if (front == -1 && rear == -1) {
      front = 0;
      rear = 0;
      q[rear] = x;
   } else {
      rear++;
      q[rear] = x;
   }
}

void dequeue() {
   if (front == -1 || front > rear) {
      printf("Underflow\n");
   } else {
      printf("Deleted: %d\n", q[front]);
      front++;
   }
}

void isempty() {
   if (front == -1 || front > rear)
      printf("Queue is empty\n");
   else
      printf("Queue is not empty\n");
}

void size() {
   if (front == -1)
      printf("Size: 0\n");
   else
      printf("Size: %d\n", rear - front + 1);
```

```
}

int main() {
   int ch, x;
   printf("Enter size of queue: ");
   scanf("%d", &n);

   create();

   while (1) {
      printf("\n1 EnQueue\n2 DeQueue\n3 IsEmpty\n4 Size\n5 Exit\n");
      printf("Enter choice: ");
      scanf("%d", &ch);

      if (ch == 1) {
         printf("Enter value: ");
         scanf("%d", &x);
         enqueue(x);
      }
      else if (ch == 2) {
         dequeue();
      }
      else if (ch == 3) {
         isempty();
      }
      else if (ch == 4) {
         size();
      }
      else if (ch == 5) {
         break;
      }
      else {
         printf("Invalid\n");
      }
   }
   return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Enter size of queue: 5

1 EnQueue
2 DeQueue
3 IsEmpty
4 Size
5 Exit
Enter choice: 1
Enter value: 10

1 EnQueue
2 DeQueue
3 IsEmpty
4 Size
5 Exit
Enter choice: 1
Enter value: 4

1 EnQueue
2 DeQueue
3 IsEmpty
4 Size
5 Exit
Enter choice: 2
Deleted: 10

1 EnQueue
2 DeQueue
3 IsEmpty
4 Size
5 Exit
Enter choice: 10
Invalid

1 EnQueue
2 DeQueue
3 IsEmpty
4 Size
5 Exit
Enter choice: 3
Queue is not empty
```

II. Design an algorithm and write a program to reverse a queue.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 14

```
#include <stdio.h>

int q[100], front = -1, rear = -1;
int st[100], top = -1;
int n;

void enqueue(int x) {
   if (rear == n-1) printf("Overflow\n");
   else {
      if (front == -1) front = 0;
      rear++;
      q[rear] = x;
   }
}

int dequeue() {
   int x = q[front];
   front++;
   if (front > rear) { front = rear = -1; }
   return x;
}

void push(int x) {
   st[++top] = x;
}

int pop() {
   return st[top--];
}

void reverseQueue() {
   while (front != -1) {
      push(dequeue());
   }
   while (top != -1) {
      enqueue(pop());
   }
}

int main() {
   int i, x;
   printf("Enter size of queue: ");
   scanf("%d", &n);
```

```
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        enqueue(x);
    }

    reverseQueue();

    printf("Reversed queue:\n");
    for (i = front; i <= rear; i++) {
        printf("%d ", q[i]);
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }

Enter size of queue: 5
Enter 5 elements:
10 20 30 40 50
Reversed queue:
50 40 30 20 10
```

III. Design an algorithm and write a program to implement Deque i.e. double ended queue. Double ended queue is a queue in which insertion and deletion can be done from both ends of the queue. The program should implement following operations: a) insertFront() - insert an element at the front of Deque b)insertEnd() - insert an element at the end of Deque c) deleteFront() - delete an element from the front of Deque d) deleteEnd() - delete an element from the end of Deque e) isEmpty() - checks whether Deque is empty or not f)isFull() - checks whether Deque is full or not g) printFront() - print Deque from front h) printEnd() - print Deque from end.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 15

```
#include <stdio.h>

int dq[50];
int front = -1, rear = -1, size;

int isEmpty() {
   return (front == -1 && rear == -1);
}

int isFull() {
   return ((front == 0 && rear == size - 1) || (front == rear + 1));
}void insertFront(int x) {
   if (isFull()) {
      printf("Deque is Full!\n");
      return;
   }
   if (isEmpty()) {
      front = rear = 0;
   } else if (front == 0) {
      front = size - 1;
   } else {
      front--;
   }
   dq[front] = x;
   printf("Inserted %d at Front.\n", x);
}void insertEnd(int x) {
   if (isFull()) {
      printf("Deque is Full!\n");
      return;
   }
   if (isEmpty()) {
      front = rear = 0;
   } else if (rear == size - 1) {
      rear = 0;
   } else {
      rear++;
   }
   dq[rear] = x;
   printf("Inserted %d at End.\n", x);
}void deleteFront() {
```

```
    if (isEmpty()) {
        printf("Deque is Empty!\n");
        return;
    }
    printf("Deleted %d from Front.\n", dq[front]);

    if (front == rear) {
        front = rear = -1;
    } else if (front == size - 1) {
        front = 0;
    } else {
        front++;
    }
}void deleteEnd() {
    if (isEmpty()) {
        printf("Deque is Empty!\n");
        return;
    }
    printf("Deleted %d from End.\n", dq[rear]);

    if (front == rear) {
        front = rear = -1;
    } else if (rear == 0) {
        rear = size - 1;
    } else {
        rear--;
    }
}void printFront() {
    if (isEmpty()) {
        printf("Deque is Empty!\n");
        return;
    }

    printf("Deque (Front to End): ");
    int i = front;
    while (1) {
        printf("%d ", dq[i]);
        if (i == rear) break;
        i = (i + 1) % size;
    }
    printf("\n");
}

void printEnd() {
    if (isEmpty()) {
        printf("Deque is Empty!\n");
        return;
    }  printf("Deque (End to Front): ");
    int i = rear;
    while (1) {
        printf("%d ", dq[i]);
```

```
        if (i == front) break;
        i = (i - 1 + size) % size;
    }
    printf("\n");
}

int main() {
    int choice, value;
    printf("Enter size of Deque: ");
    scanf("%d", &size);

    while (1) {
        printf("\n--- DEQUE MENU ---\n");
        printf("1. Insert Front\n2. Insert End\n3. Delete Front\n4. Delete End\n");
        printf("5. Check Empty\n6. Check Full\n7. Print Front\n8. Print End\n9. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &value); insertFront(value); break;
            case 2: printf("Enter value: "); scanf("%d", &value); insertEnd(value); break;
            case 3: deleteFront(); break;
            case 4: deleteEnd(); break;
            case 5: printf(isEmpty() ? "Deque is Empty\n" : "Deque is NOT Empty\n"); break;
            case 6: printf(isFull() ? "Deque is Full\n" : "Deque is NOT Full\n"); break;
            case 7: printFront(); break;
            case 8: printEnd(); break;
            case 9: return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Enter size of Deque: 5

--- DEQUE MENU ---
1. Insert Front
2. Insert End
3. Delete Front
4. Delete End
5. Check Empty
6. Check Full
7. Print Front
8. Print End
9. Exit
Enter your choice: 1
Enter value: 10
Inserted 10 at Front.

--- DEQUE MENU ---
1. Insert Front
2. Insert End
3. Delete Front
4. Delete End
5. Check Empty
6. Check Full
7. Print Front
8. Print End
9. Exit
Enter your choice: 2
Enter value: 20
Inserted 20 at End.

--- DEQUE MENU ---
1. Insert Front
2. Insert End
3. Delete Front
4. Delete End
5. Check Empty
6. Check Full
7. Print Front
8. Print End
9. Exit
Enter your choice: 7
Deque (Front to End): 10 20

--- DEQUE MENU ---
1. Insert Front
2. Insert End
3. Delete Front
```

4. Delete End
5. Check Empty
6. Check Full
7. Print Front
8. Print End
9. Exit
Enter your choice: 4
Deleted 20 from End.

WEEK6:
Design an algorithm and write a program to implement stack operations using minimum number of queues.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 16

```
#include <stdio.h>

int queue[50];
int front = -1, rear = -1, size;

int isEmpty() {
   return (front == -1);
}

void enqueue(int x) {
   if ((rear + 1) % size == front) {
      printf("Queue/Stack Overflow!\n");
      return;
   }
   if (isEmpty()) {
      front = rear = 0;
   } else {
      rear = (rear + 1) % size;
   }
   queue[rear] = x;
}

int dequeue() {
   if (isEmpty()) {
      printf("Stack is Empty!\n");
      return -1;
   }

   int val = queue[front];

   if (front == rear) {
      front = rear = -1;
   } else {
      front = (front + 1) % size;
   }

   return val;
}

void push(int x) {
   enqueue(x);

   int i, count = (rear >= front) ? (rear - front) : (size - front + rear);
```

```
    for (i = 0; i < count; i++) {
        enqueue(dequeue());
    }

    printf("Pushed: %d\n", x);
}

void pop() {
    if (isEmpty()) {
        printf("Stack Underflow!\n");
        return;
    }
    int val = dequeue();
    printf("Popped: %d\n", val);
}

void peek() {
    if (isEmpty()) {
        printf("Stack is Empty!\n");
        return;
    }
    printf("Top Element: %d\n", queue[front]);
}

void display() {
    if (isEmpty()) {
        printf("Stack is Empty!\n");
        return;
    }

    printf("Stack (top to bottom): ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear) break;
        i = (i + 1) % size;
    }
    printf("\n");
}

int main() {
    int choice, value;
    printf("Enter size of Stack: ");
    scanf("%d", &size);

    printf("\n--- Stack using 1 Queue ---\n");
    while (1) {
        printf("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
```

```
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Enter size of Stack: 5

--- Stack using 1 Queue ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 10
Pushed: 10

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 20
Pushed: 20

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 30
Pushed: 30

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 4
Stack (top to bottom): 30 20 10

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 2
Popped: 30
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 3
Top Element: 20

II. Design an algorithm and write a program to implement queue operations using minimum number of stacks.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 17

```
#include <stdio.h>

int s1[100], s2[100];
int top1 = -1, top2 = -1;

void push1(int x) {
   s1[++top1] = x;
}

int pop1() {
   return s1[top1--];
}

void push2(int x) {
   s2[++top2] = x;
}

int pop2() {
   return s2[top2--];
}

int isEmpty() {
   return (top1 == -1 && top2 == -1);
}

void enqueue(int x) {
   push1(x);
}

int dequeue() {
   int val;
   if (isEmpty()) {
      printf("Queue is empty\n");
      return -1;
   }

   if (top2 == -1) {
      while (top1 != -1) {
         push2(pop1());
      }
   }

   val = pop2();
```

```
    return val;
}

void display() {
    int i;
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }

    if (top2 == -1) {
        for (i = 0; i <= top1; i++)
            printf("%d ", s1[i]);
    } else {
        for (i = top2; i >= 0; i--)
            printf("%d ", s2[i]);
        for (i = 0; i <= top1; i++)
            printf("%d ", s1[i]);
    }
    printf("\n");
}

int main() {
    int n, x, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        enqueue(x);
    }

    printf("Queue: ");
    display();

    printf("Dequeued: %d\n", dequeue());
    printf("Dequeued: %d\n", dequeue());

    printf("Queue after deletions: ");
    display();

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Enter number of elements: 5
Enter elements:
10 20 30 40 50
Queue: 10 20 30 40 50
Dequeued: 10
Dequeued: 20
Queue after deletions: 30 40 50
```

III. Design an algorithm and write a program to implement circular queue. Circular queue is a queue in which last position of queue is connected with first position of queue to make a circle. The program should implement following operations: a) Create() - create a queue of specific size b)EnQueue(k) - insert an element k into the queue c) DeQueue() - delete an element from the queue d)IsEmpty() - check if queue is empty or not e) Front() - return front item from queue f) Rear() - return rear item from queue.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 18

```
#include <stdio.h>

int queue[50];
int front = -1, rear = -1, size;

void create(int n) {
   size = n;
   front = rear = -1;
}

int isEmpty() {
   return (front == -1);
}

void enqueue(int x) {
   if ((rear + 1) % size == front) {
      printf("Queue is full\n");
      return;
   }
   if (front == -1) {
      front = rear = 0;
      queue[rear] = x;
   } else {
      rear = (rear + 1) % size;
      queue[rear] = x;
   }
}

int dequeue() {
   if (isEmpty()) {
      printf("Queue is empty\n");
      return -1;
   }
   int val = queue[front];
   if (front == rear) {
      front = rear = -1;
   } else {
      front = (front + 1) % size;
   }
   return val;
```

```
}

int frontElement() {
   if (!isEmpty())
      return queue[front];
   return -1;
}

int rearElement() {
   if (!isEmpty())
      return queue[rear];
   return -1;
}

int main() {
   int n, x, i;
   printf("Enter size of queue: ");
   scanf("%d", &n);
   create(n);

   printf("Enter %d elements:\n", n-1);
   for (i = 0; i < n-1; i++) {
      scanf("%d", &x);
      enqueue(x);
   }

   printf("Front element: %d\n", frontElement());
   printf("Rear element: %d\n", rearElement());

   printf("Dequeued: %d\n", dequeue());
   printf("Dequeued: %d\n", dequeue());

   printf("Front after dequeue: %d\n", frontElement());
   printf("Rear after dequeue: %d\n", rearElement());

   return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter size of queue: 5
Enter 4 elements:
10 20 30 40 50
Front element: 10
Rear element: 40
Dequeued: 10
Dequeued: 20
Front after dequeue: 30
Rear after dequeue: 40
```

WEEK7:
I. Write an algorithm and a program to implement singly linked list. The program should implement following operations: a) Create(k) - create a linked list with single node of value k b)InsertFront(k) - insert node of value k at the front of the linked list c) InsertEnd(k) - insert a node of value k at the end of the linked list d)InsertAnywhere(p) - insert a node at specific position p e) DeleteFront() - delete a node from the front of the linked list f) DeleteEnd() - delete a node from the end of the linked list g)DeleteAnywhere(p) - delete a node from a specific position p h)Size() - find the size of the linked list i) IsEmpty() - checks if the linked list is empty or not j) FindMiddle() - finds the middle element of the linked list.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 19

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* head = NULL;

void create(int k) {
   head = (struct Node*)malloc(sizeof(struct Node));
   head->data = k;
   head->next = NULL;
}

void insertFront(int k) {
   struct Node* new = (struct Node*)malloc(sizeof(struct Node));
   new->data = k;
   new->next = head;
   head = new;
}

void insertEnd(int k) {
   struct Node* new = (struct Node*)malloc(sizeof(struct Node));
   new->data = k;
   new->next = NULL;

   if (head == NULL) {
      head = new;
      return;
   }

   struct Node* temp = head;
   while (temp->next != NULL)
      temp = temp->next;
   temp->next = new;
}
```

```
void insertAnywhere(int k, int p) {
   if (p == 1) {
      insertFront(k);
      return;
   }

   struct Node* new = (struct Node*)malloc(sizeof(struct Node));
   new->data = k;

   struct Node* temp = head;
   int i;
   for (i = 1; i < p-1 && temp != NULL; i++)
      temp = temp->next;

   if (temp == NULL) return;

   new->next = temp->next;
   temp->next = new;
}

void deleteFront() {
   if (head == NULL) return;

   struct Node* temp = head;
   head = head->next;
   free(temp);
}

void deleteEnd() {
   if (head == NULL) return;

   struct Node* temp = head;
   if (temp->next == NULL) {
      head = NULL;
      free(temp);
      return;
   }

   while (temp->next->next != NULL)
      temp = temp->next;

   free(temp->next);
   temp->next = NULL;
}

void deleteAnywhere(int p) {
   if (p == 1) {
      deleteFront();
      return;
   }
```

```
    struct Node* temp = head;
    int i;
    for (i = 1; i < p-1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) return;

    struct Node* del = temp->next;
    temp->next = del->next;
    free(del);
}

int size() {
    int count = 0;
    struct Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

int isEmpty() {
    return (head == NULL);
}

int findMiddle() {
    struct Node* slow = head;
    struct Node* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}

void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    create(10);
    insertEnd(20);
    insertEnd(30);
    insertFront(5);
```

```
    insertAnywhere(25,3);

    printf("List: ");
    display();

    deleteFront();
    deleteEnd();
    deleteAnywhere(2);

    printf("List after deletions: ");
    display();

    printf("Size: %d\n", size());
    printf("IsEmpty: %d\n", isEmpty());
    printf("Middle: %d\n", findMiddle());

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
List: 5 10 25 20 30
List after deletions: 10 20
Size: 2
IsEmpty: 0
Middle: 20
```

II. Write an algorithm and a program to implement queue using linked list. The program should implement following stack operations: a) Create() b) EnQueue() c) DeQueue() d) IsEmpty() e) Size().
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 20

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

void create() {
    front = rear = NULL;
}

int isEmpty() {
    return (front == NULL);
}

void enqueue(int x) {
    struct Node* new = (struct Node*)malloc(sizeof(struct Node));
    new->data = x;
    new->next = NULL;

    if (front == NULL) {
        front = rear = new;
    } else {
        rear->next = new;
        rear = new;
    }
}

void dequeue() {
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
    struct Node* temp = front;
    printf("Dequeued: %d\n", temp->data);
    front = front->next;

    if (front == NULL)
        rear = NULL;
```

```
    free(temp);
}

int size() {
    int count = 0;
    struct Node* temp = front;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

void display() {
    struct Node* temp = front;
    printf("Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    create();

    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);

    display();

    dequeue();
    dequeue();

    display();

    printf("IsEmpty: %d\n", isEmpty());
    printf("Size: %d\n", size());

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Queue: 10 20 30 40
Dequeued: 10
Dequeued: 20
Queue: 30 40
IsEmpty: 0
Size: 2
```

III. Write an algorithm and a program to implement stack using linked list. he program should implement following queue operations: a) Create() b) Push() c) Pop() d) IsEmpty() e) IsFull() f) Size()
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO – 21

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* top = NULL;

void create() {
   top = NULL;
}

int isEmpty() {
   if(top == NULL)
      return 1;
   else
      return 0;
}

int isFull() {
   struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
   if(temp == NULL)
      return 1;
   else {
      free(temp);
      return 0;
   }
}

void push(int value) {
   if(isFull()) {
      printf("Stack is Full\n");
      return;
   }

   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = top;
   top = newNode;
   printf("Pushed %d\n", value);
}
```

```
void pop() {
   if(isEmpty()) {
      printf("Stack is Empty\n");
      return;
   }

   struct Node* temp = top;
   printf("Popped %d\n", top->data);
   top = top->next;
   free(temp);
}

int size() {
   int count = 0;
   struct Node* temp = top;
   while(temp != NULL) {
      count++;
      temp = temp->next;
   }
   return count;
}int main() {
   int choice, value;
   create();
  while(1) {
      printf("\n1 Push");
      printf("\n2 Pop");
      printf("\n3 IsEmpty");
      printf("\n4 IsFull");
      printf("\n5 Size");
      printf("\n6 Exit");
      printf("\nEnter choice: ");
      scanf("%d", &choice);

      if(choice == 1) {
         printf("Enter value: ");
         scanf("%d", &value);
         push(value);
      }
      else if(choice == 2) {
         pop();
      }
      else if(choice == 3) {
         if(isEmpty())
            printf("Stack is Empty\n");
         else
            printf("Stack is Not Empty\n");
      }
      else if(choice == 4) {
         if(isFull())
            printf("Stack is Full\n");
```

```
            else
                printf("Stack is Not Full\n");
        }
        else if(choice == 5) {
            printf("Size = %d\n", size());
        }
        else if(choice == 6) {
            break;
        }
        else {
            printf("Invalid choice\n");
        }
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
1 Push
2 Pop
3 IsEmpty
4 IsFull
5 Size
6 Exit
Enter choice: 1
Enter value: 10
Pushed 10
```

WEEK8:

I. Write an algorithm and a program to implement doubly linked list. The program should implement following operations: a) Create(k) - create a doubly linked list node with value k. b)InsertFront(k) - insert node at the front of the linked list. c) InsertEnd(k) - insert a node at the end of the linked list. d)InsertIntermediate(k,p) - insert a node at specific position p. e) DeleteFront() - delete a node from the front of the linked list. f) DeleteEnd() - delete a node from the end of the linked list. g)DeleteIntermediate(p) – delete a node from a specific position p. h)Size() - returns the size of doubly linked list. i) IsEmpty() - checks whether the list is empty or not. j) FindMiddle() - returns the contents of middle node of the list.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 22

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};struct Node* head = NULL;

int size() {
   int count = 0;
   struct Node* temp = head;
   while(temp != NULL) {
      count++;
      temp = temp->next;
   }
   return count;
}

int isEmpty() {
   if(head == NULL)
      return 1;
   else
      return 0;
}void create(int k) {
   head = (struct Node*)malloc(sizeof(struct Node));
   head->data = k;
   head->next = NULL;
   head->prev = NULL;
}void insertFront(int k) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = k;
   newNode->prev = NULL;
   newNode->next = head;
   if(head != NULL)
      head->prev = newNode;
   head = newNode;
```

```
}void insertEnd(int k) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = k;
   newNode->next = NULL;

   if(head == NULL) {
      newNode->prev = NULL;
      head = newNode;
      return;
   }

   struct Node* temp = head;
   while(temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
   newNode->prev = temp;
}

void insertIntermediate(int k, int p) {
   if(p == 1) {
      insertFront(k);
      return;
   }struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = k;

   struct Node* temp = head;
   for(int i = 1; i < p-1 && temp != NULL; i++)
      temp = temp->next;

   if(temp == NULL)
      return;

   newNode->next = temp->next;
   if(temp->next != NULL)
      temp->next->prev = newNode;

   temp->next = newNode;
   newNode->prev = temp;
}

void deleteFront() {
   if(head == NULL)
      return;

   struct Node* temp = head;
   head = head->next;
   if(head != NULL)
      head->prev = NULL;

   free(temp);
```

```
}

void deleteEnd() {
   if(head == NULL)
      return;

   struct Node* temp = head;

   if(temp->next == NULL) {
      free(head);
      head = NULL;
      return;
   }

   while(temp->next != NULL)
      temp = temp->next;

   temp->prev->next = NULL;
   free(temp);
}

void deleteIntermediate(int p) {
   if(p == 1) {
      deleteFront();
      return;
   }

   struct Node* temp = head;
   for(int i = 1; i < p && temp != NULL; i++)
      temp = temp->next;

   if(temp == NULL)
      return;

   if(temp->next != NULL)
      temp->next->prev = temp->prev;

   temp->prev->next = temp->next;

   free(temp);
}

void findMiddle() {
   if(head == NULL) {
      printf("List is empty\n");
      return;
   }

   struct Node* slow = head;
   struct Node* fast = head;
```

```
    while(fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    printf("Middle Element: %d\n", slow->data);
}

void display() {
    struct Node* temp = head;
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    int ch, val, pos;

    while(1) {
        printf("\n1 Create");
        printf("\n2 InsertFront");
        printf("\n3 InsertEnd");
        printf("\n4 InsertIntermediate");
        printf("\n5 DeleteFront");
        printf("\n6 DeleteEnd");
        printf("\n7 DeleteIntermediate");
        printf("\n8 Size");
        printf("\n9 IsEmpty");
        printf("\n10 FindMiddle");
        printf("\n11 Display");
        printf("\n12 Exit");

        printf("\nEnter choice: ");
        scanf("%d", &ch);

        if(ch == 1) {
            printf("Enter value: ");
            scanf("%d", &val);
            create(val);
        }
        else if(ch == 2) {
            printf("Enter value: ");
            scanf("%d", &val);
            insertFront(val);
        }
        else if(ch == 3) {
            printf("Enter value: ");
            scanf("%d", &val);
            insertEnd(val);
        }
```

```
        else if(ch == 4) {
            printf("Enter value and position: ");
            scanf("%d %d", &val, &pos);
            insertIntermediate(val, pos);
        }
        else if(ch == 5) {
            deleteFront();
        }
        else if(ch == 6) {
            deleteEnd();
        }
        else if(ch == 7) {
            printf("Enter position: ");
            scanf("%d", &pos);
            deleteIntermediate(pos);
        }
        else if(ch == 8) {
            printf("Size: %d\n", size());
        }
        else if(ch == 9) {
            if(isEmpty())
                printf("List is empty\n");
            else
                printf("List not empty\n");
        }
        else if(ch == 10) {
            findMiddle();
        }
        else if(ch == 11) {
            display();
        }
        else if(ch == 12) {
            break;
        }
        else {
            printf("Invalid\n");
        }
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 1
Enter value: 10

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 2
Enter value: 5

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 3
Enter value: 20

1 Create
2 InsertFront
3 InsertEnd
```

4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 4
Enter value and position: 15 2

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 11
5 15 10 20
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 10
Middle Element: 10

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle

11 Display
12 Exit
Enter choice: 8
Size: 4

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 5

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 11
15 10 20
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 7
Enter position: 2
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate

5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 11
15 20
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 9
List not empty
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 10
Middle Element: 20
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 FindMiddle
11 Display
12 Exit
Enter choice: 12

II. Given a doubly linked list, design an algorithm and write a program to reverse this list without using any extra space.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 23

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};struct Node* head = NULL;

void insertEnd(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = NULL;
   newNode->prev = NULL;

   if(head == NULL) {
      head = newNode;
      return;
   }struct Node* temp = head;
   while(temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
   newNode->prev = temp;
}void display() {
   struct Node* temp = head;
   while(temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }
}void reverseDLL() {
   struct Node* current = head;
   struct Node* temp = NULL;

   while(current != NULL) {
      temp = current->prev;
      current->prev = current->next;
      current->next = temp;
      current = current->prev;
   }if(temp != NULL)
      head = temp->prev;
}int main() {
   int n, value;

   printf("Enter number of nodes: ");
```

```
    scanf("%d", &n);

    printf("Enter values:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(value);
    }

    printf("Original List: ");
    display();

    reverseDLL();

    printf("\nReversed List: ");
    display();

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }

Enter number of nodes: 5
Enter values:
10 20 30 40 50
Original List: 10 20 30 40 50
Reversed List: 50 40 30 20 10
```

III. . Given a sorted doubly linked list, design an algorithm and write a program to eliminate duplicates from this list.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 24

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};struct Node* head = NULL;

void insertEnd(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = NULL;
   newNode->prev = NULL;

   if(head == NULL) {
      head = newNode;
      return;
   }  struct Node* temp = head;
   while(temp->next != NULL)
      temp = temp->next;temp->next = newNode;
   newNode->prev = temp;
}void removeDuplicates() {
   struct Node* current = head;

   if(head == NULL)
      return;while(current->next != NULL) {
      if(current->data == current->next->data) {
         struct Node* dup = current->next;
         current->next = dup->next;

         if(dup->next != NULL)
            dup->next->prev = current;

         free(dup);
      } else {
         current = current->next;
      }
   }
}

void display() {
   struct Node* temp = head;
   while(temp != NULL) {
      printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
}int main() {
    int n, val;
    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter sorted values:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertEnd(val);
    }printf("Original List: ");
    display();

    removeDuplicates();

    printf("\nList after removing duplicates: ");
    display();

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of nodes: 7
Enter sorted values:
2 3 4 5 6 7 8
Original List: 2 3 4 5 6 7 8
List after removing duplicates: 2 3 4 5 6 7 8
```

WEEK9:
I. Design an algorithm and write a program to implement circular linked list. The program should implement following operations: a) Create(k) - creates a circular linked list node with value k. b)InsertFront() - insert node at the front of list. c) InsertEnd() - insert node at the end of list. d)InsertIntermediate() - insert node at any specified position of list. e) DeleteFront() - delete node from the front of list. f) DeleteEnd() - delete node from the end of list. g)DeleteIntermediate() - delete node from any specified position of list. h)Size() - return size of circular linked list. i) IsEmpty() - checks whether list is empty or not.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 25

```
#include <stdio.h>#include <stdlib.h>
struct Node {
   int data;
   struct Node* next;
};struct Node* head = NULL;
int isEmpty() {
   return head == NULL;
}int size() {
   if(head == NULL) return 0;
   struct Node* temp = head;
   int count = 0;
   do {
      count++;
      temp = temp->next;
   } while(temp != head);
   return count;
}void create(int k) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = k;
   newNode->next = newNode;
   head = newNode;
}void insertFront(int k) {
   if(isEmpty()) {
      create(k);
      return;
   } struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = k;

   struct Node* temp = head;
   while(temp->next != head)
      temp = temp->next;

   newNode->next = head;
   temp->next = newNode;
   head = newNode;
}void insertEnd(int k) {
   if(isEmpty()) {
      create(k);
      return;
```

```
    } struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = k;
    struct Node* temp = head;

    while(temp->next != head)
        temp = temp->next;

    temp->next = newNode;
    newNode->next = head;
}void insertIntermediate(int k, int p) {
    if(p == 1) {
        insertFront(k);
        return;
    }  struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = k;

    struct Node* temp = head;

    for(int i = 1; i < p-1 && temp->next != head; i++)
        temp = temp->next;

    newNode->next = temp->next;
    temp->next = newNode;
}void deleteFront() {
    if(isEmpty()) return;

    if(head->next == head) {
        free(head);
        head = NULL;
        return;
    }  struct Node* temp = head;
    struct Node* last = head;
    while(last->next != head)
        last = last->next;  head = head->next;
    last->next = head;
    free(temp);
}void deleteEnd() {
    if(isEmpty()) return;
struct Node* temp = head;
 if(head->next == head) {
        free(head);
        head = NULL;
        return;
    } struct Node* prev = NULL;
    while(temp->next != head) {
        prev = temp;
        temp = temp->next;} prev->next = head;
    free(temp);
}void deleteIntermediate(int p) {
    if(p == 1) {
        deleteFront();
```

```
      return;
   } struct Node* temp = head;
   struct Node* prev = NULL;

   for(int i = 1; i < p && temp->next != head; i++) {
      prev = temp;
      temp = temp->next;
   }prev->next = temp->next;
   free(temp);
}void display() {
   if(isEmpty()) {
      printf("List empty\n");
      return;
   }struct Node* temp = head;
   do {
      printf("%d ", temp->data);
      temp = temp->next;
   } while(temp != head);
}int main() {
   int ch, val, pos;

   while(1) {
      printf("\n1 Create");
      printf("\n2 InsertFront");
      printf("\n3 InsertEnd");
      printf("\n4 InsertIntermediate");
      printf("\n5 DeleteFront");
      printf("\n6 DeleteEnd");
      printf("\n7 DeleteIntermediate");
      printf("\n8 Size");
      printf("\n9 IsEmpty");
      printf("\n10 Display");
      printf("\n11 Exit");

      printf("\nEnter choice: ");
      scanf("%d", &ch);

      if(ch == 1) {
         printf("Enter value: ");
         scanf("%d", &val);
         create(val);
      }
      else if(ch == 2) {
         printf("Enter value: ");
         scanf("%d", &val);
         insertFront(val);
      }
      else if(ch == 3) {
         printf("Enter value: ");
         scanf("%d", &val);
         insertEnd(val);
```

```
        }
        else if(ch == 4) {
            printf("Enter value and position: ");
            scanf("%d %d", &val, &pos);
            insertIntermediate(val, pos);
        }
        else if(ch == 5) deleteFront();
        else if(ch == 6) deleteEnd();
        else if(ch == 7) {
            printf("Enter position: ");
            scanf("%d", &pos);
            deleteIntermediate(pos);
        }
        else if(ch == 8) printf("Size: %d\n", size());
        else if(ch == 9) {
            if(isEmpty()) printf("Empty\n");
            else printf("Not Empty\n");
        }
        else if(ch == 10) {
            display();
        }
        else if(ch == 11) break;
        else printf("Invalid choice\n");
    }
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 1
Enter value: 10

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 3
Enter value: 20

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 2
Enter value: 5

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
```

8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 4
Enter value and position: 15 2

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 10
5 15 10 20
1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 8
Size: 4

1 Create
2 InsertFront
3 InsertEnd
4 InsertIntermediate
5 DeleteFront
6 DeleteEnd
7 DeleteIntermediate
8 Size
9 IsEmpty
10 Display
11 Exit
Enter choice: 0
Invalid choice

II. Design an algorithm and write a program to concatenate two circularly linked lists.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 26

```
#include <stdio.h>
#include <stdlib.h>struct Node {
   int data;
   struct Node *next;
};struct Node *head1 = NULL, *head2 = NULL;
void insertEnd(struct Node **head, int value) {
   struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
   newNode->data = value;
if (*head == NULL) {
      *head = newNode;
      newNode->next = *head;
      return;
   } struct Node *temp = *head;
   while (temp->next != *head)
      temp = temp->next;
temp->next = newNode;
   newNode->next = *head;
}struct Node* concatenate(struct Node *h1, struct Node *h2) {
   if (h1 == NULL) return h2;If (h2 == NULL) return h1; struct Node *t1 = h1, *t2 = h2;
 while (t1->next != h1) t1 = t1->next;
   while (t2->next != h2) t2 = t2->next;

   t1->next = h2;
   t2->next = h1;

   return h1;
}// Display list
void display(struct Node *head) {
   if (head == NULL) {
      printf("List is empty\n");
      return;
   }struct Node *temp = head;
   do {
      printf("%d ", temp->data);
      temp = temp->next;
   } while (temp != head);
   printf("\n");
}int main() {
   int n1, n2, val;printf("Enter number of nodes in List 1: ");
   scanf("%d", &n1);
   for (int i = 0; i < n1; i++) {
      printf("Enter value: ");
      scanf("%d", &val);
      insertEnd(&head1, val);
   }
```

```
    printf("Enter number of nodes in List 2: ");
    scanf("%d", &n2);
    for (int i = 0; i < n2; i++) {
        printf("Enter value: ");
        scanf("%d", &val);
        insertEnd(&head2, val);
    }

    printf("\nList 1: ");
    display(head1);
    printf("List 2: ");
    display(head2);

    head1 = concatenate(head1, head2);

    printf("\nConcatenated Circular Linked List: ");
    display(head1);
return 0;}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Enter number of nodes in List 1: 3
Enter value: 10
Enter value: 20
Enter value: 30
Enter number of nodes in List 2: 2
Enter value: 12
Enter value: 13

List 1: 10 20 30
List 2: 12 13

Concatenated Circular Linked List: 10 20 30 12 13
```

III. Write an algorithm and a program that will split a circularly linked list into two circularly linked list provided position from where circular linked list has to be splitted.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 27

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};struct Node *head = NULL, *head2 = NULL;

void insertEnd(int val) {
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;

   if (head == NULL) {
      head = newNode;
      newNode->next = head;
      return;
   }  struct Node *temp = head;
   while (temp->next != head)
      temp = temp->next;

   temp->next = newNode;
   newNode->next = head;
}void display(struct Node *h) {
   if (h == NULL) {
      printf("List is empty\n");
      return;
   }
   struct Node *temp = h;
   do {
      printf("%d ", temp->data);
      temp = temp->next;
   } while (temp != h);
   printf("\n");
}void splitList(int pos) {
   if (head == NULL) return;
  struct Node *temp = head;
   int i = 1;

   while (i < pos && temp->next != head) {
      temp = temp->next;
      i++;
   } head2 = temp->next;
   temp->next = head;
  struct Node *temp2 = head2;
   while (temp2->next != head)
```

```
        temp2 = temp2->next;

    temp2->next = head2;
}
int main() {
    int n, val, pos;
 printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &val);
        insertEnd(val);
    } printf("Enter position to split: ");
    scanf("%d", &pos);
splitList(pos); printf("\nFirst Circular Linked List:\n");
    display(head);printf("Second Circular Linked List:\n");
    display(head2);
return 0;}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of nodes: 5
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value: 50
Enter position to split: 3

First Circular Linked List:
10 20 30
Second Circular Linked List:
40 50
```

WEEK10:
I. Write an algorithm that will split a linked list into two linked lists, so that successive nodes go to different lists. Odd numbered nodes to the first list while even numbered nodes to the second list.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 28

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};struct Node* head = NULL;
struct Node* head1 = NULL; // odd list
struct Node* head2 = NULL; // even list

void insertEnd(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
      return;
   } struct Node* temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
}void splitList() {
   struct Node* temp = head;
   struct Node *oddTail = NULL, *evenTail = NULL;
   int pos = 1;

   while (temp != NULL) {
      if (pos % 2 != 0) {
         if (head1 == NULL) {
            head1 = temp;
            oddTail = temp;
         } else {
            oddTail->next = temp;
            oddTail = temp;
         }
      } else {
         if (head2 == NULL) {
            head2 = temp;
            evenTail = temp;
         } else {
            evenTail->next = temp;
```

```
            evenTail = temp;
        }
      }
      temp = temp->next;
      pos++;
   } if (oddTail != NULL) oddTail->next = NULL;
   if (evenTail != NULL) evenTail->next = NULL;
}void display(struct Node* h) {
   if (h == NULL) {
      printf("Empty\n");
      return;
   }

   while (h != NULL) {
      printf("%d ", h->data);
      h = h->next;
   }
   printf("\n");
}int main() {
   int n, val;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (int i = 0; i < n; i++) {
      printf("Enter value: ");
      scanf("%d", &val);
      insertEnd(val);
   }splitList();
printf("\nOriginal List: ");
   display(head);
 printf("Odd Position List: ");
   display(head1);
printf("Even Position List: ");
   display(head2); return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of nodes: 6
Enter value: 10
Enter value: 12
Enter value: 13
Enter value: 45
Enter value: 67
Enter value: 87

Original List: 10 13 67
Odd Position List: 10 13 67
Even Position List: 12 45 87
```

II. Given a linked list and a number n, design an algorithm and write a program to find the value at the n'th node from end of this linked list.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 29

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* head = NULL;

void insertEnd(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
      return;
   }

   struct Node* temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
}

int findNthFromEnd(int n) {
   struct Node *p = head, *q = head;
   int count = 0;

   while (count < n) {
      if (p == NULL)
         return -1;     // n > list size
      p = p->next;
      count++;
   }

   while (p != NULL) {
      p = p->next;
      q = q->next;
   }
```

```
    return q->data;
}

int main() {
    int n, val, nodes;

    printf("Enter number of nodes: ");
    scanf("%d", &nodes);

    for (int i = 0; i < nodes; i++) {
        printf("Enter value: ");
        scanf("%d", &val);
        insertEnd(val);
    }

    printf("Enter n (position from end): ");
    scanf("%d", &n);

    int result = findNthFromEnd(n);

    if (result == -1)
        printf("Invalid position\n");
    else
        printf("Value at %dth node from end = %d\n", n, result);

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of nodes: 5
Enter value: 12
Enter value: 54
Enter value: 67
Enter value: 34
Enter value: 21
Enter n (position from end): 3
Value at 3th node from end = 67
```

III. Write a program that will reverse a linked list only by traversing it once and without using extra space.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 30

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* head = NULL;

void insertEnd(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
      return;
   }

   struct Node* temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
}

void reverseList() {
   struct Node *prev = NULL, *curr = head, *next = NULL;

   while (curr != NULL) {
      next = curr->next;
      curr->next = prev;
      prev = curr;
      curr = next;
   }

   head = prev;
}

void display() {
   struct Node* temp = head;
   while (temp != NULL) {
      printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &val);
        insertEnd(val);
    }

    printf("\nOriginal List: ");
    display();
   reverseList();

    printf("Reversed List: ");
    display();

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }

Enter number of nodes: 5
Enter value: 54
Enter value: 56
Enter value: 78
Enter value: 33
Enter value: 23

Original List: 54 56 78 33 23
Reversed List: 23 33 78 56 54
```

WEEK11:
I. Design an algorithm to implement binary trees. Write a program which implements following basic operations on binary tree: a) CreateTree() - creates a root node b) InsertNode() - insert a node in tree c) DeleteNode() - delete a node from tree d) FindHeight() - find the height of tree e) FindSize() - find number of nodes in tree f) Search() - search whether given specific element present in tree or not.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 31

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int val) {
   struct Node* t = (struct Node*)malloc(sizeof(struct Node));
   t->data = val;
   t->left = t->right = NULL;
   return t;
}

struct Node* root = NULL;
struct Node* arrNodes[100];

void CreateTree(int arr[], int n) {
   for (int i = 0; i < n; i++)
      arrNodes[i] = NULL;

   for (int i = 0; i < n; i++)
      if (arr[i] != -1)
         arrNodes[i] = createNode(arr[i]);

   for (int i = 0; i < n; i++) {
      if (arrNodes[i] != NULL) {
         int l = 2*i + 1;
         int r = 2*i + 2;

         if (l < n && arrNodes[l] != NULL)
            arrNodes[i]->left = arrNodes[l];
         if (r < n && arrNodes[r] != NULL)
            arrNodes[i]->right = arrNodes[r];
      }
   }
   root = arrNodes[0];
}
```

```
int height(struct Node* r) {
    if (r == NULL) return 0;
    int lh = height(r->left);
    int rh = height(r->right);
    return (lh > rh ? lh : rh) + 1;
}

int size(struct Node* r) {
    if (r == NULL) return 0;
    return 1 + size(r->left) + size(r->right);
}

int search(struct Node* r, int key) {
    if (r == NULL) return 0;
    if (r->data == key) return 1;
    return search(r->left, key) || search(r->right, key);
}

int main() {
    int n, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter tree elements (-1 for NULL): ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    CreateTree(arr, n);

    printf("Tree height = %d\n", height(root));
    printf("Tree size = %d\n", size(root));

    printf("Enter value to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Found\n");
    else
        printf("Not found\n");

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of elements: 15
Enter tree elements (-1 for NULL): 1 2 3 4 -1 5 6 7 8 -1 -1 9 -1 -1 -1
Tree height = 4
Tree size = 9
Enter value to search: 9
Found
```

II. Write an algorithm and a program to implement following types of traversing in the tree: a) inorder() - (1) traverse left subtree, (2) traverse root, (3) traverse right subtree b) postorder() - (1) traverse root, (2) traverse left subtree, (3) traverse right subtree c) preorder() - (1) traverse left subtree, (2) traverse right subtree, (3) traverse root.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 32

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void preorder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void postorder(struct Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

int main() {
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
```

```
    root->left->right = createNode(5);
    root->right->right = createNode(6);

    printf("Inorder Traversal: ");
    inorder(root);

    printf("\nPreorder Traversal: ");
    preorder(root);

    printf("\nPostorder Traversal: ");
    postorder(root);

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11 } ; if ($?) { .\que11 }

Inorder Traversal: 4 2 5 1 3 6
Preorder Traversal: 1 2 4 5 3 6
Postorder Traversal: 4 5 2 6 3
```

III. For an expression in the form of binary tree, infix representation of the expression is given in the form of character array.. Design an algorithm and a program to convert infix expression of this tree to postfix expression. Postfix representation of expression should follow BODMAS rule.

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 33

```
#include <stdio.h>

char stack[50];
int top = -1;

void push(char x) {
   stack[++top] = x;
}

char pop() {
   return stack[top--];
}

int precedence(char x) {
   if (x == '^') return 3;
   if (x == '*' || x == '/') return 2;
   if (x == '+' || x == '-') return 1;
   return 0;
}

int main() {
   char infix[50], postfix[50];
   int i = 0, j = 0;
   char x, ch;

   printf("Enter infix expression: ");
   scanf("%s", infix);

   while ((ch = infix[i++]) != '\0') {
      if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z') || (ch >= '0' && ch <= '9'))
         postfix[j++] = ch;
      else if (ch == '(')
         push(ch);
      else if (ch == ')') {
         while (top != -1 && stack[top] != '(')
            postfix[j++] = pop();
         pop();
      }
      else {
         while (top != -1 && precedence(stack[top]) >= precedence(ch))
            postfix[j++] = pop();
         push(ch);
      }
```

```
    }

    while (top != -1)
        postfix[j++] = pop();

    postfix[j] = '\0';

    printf("Postfix Expression: %s", postfix);
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter infix expression: A+B*C-D
Postfix Expression: ABC*+D-
```

Week 12: I. Design an algorithm and a program to implement binary search tree. The program should implement following BST operations: a) Create() - creates a root node. b) Insert() - insert a node in tree. c) Delete() - delete a node from tree. d) FindHeight() - return height of tree. e) FindDepth() - return depth of tree. f) Search() - search whether given key is present in tree or not.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 34

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return createNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else if (val > root->data) root->right = insert(root->right, val);
    return root;
}

struct Node* findMin(struct Node* node) {
    while (node && node->left != NULL) node = node->left;
    return node;
}

struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;
    if (key < root->data) root->left = deleteNode(root->left, key);
    else if (key > root->data) root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL) {
            struct Node* t = root->right;
            free(root);
            return t;
        }
        else if (root->right == NULL) {
            struct Node* t = root->left;
            free(root);
```

```
            return t;
        }
        struct Node* t = findMin(root->right);
        root->data = t->data;
        root->right = deleteNode(root->right, t->data);
    }
    return root;
}

int search(struct Node* root, int key) {
    if (root == NULL) return 0;
    if (key < root->data) return search(root->left, key);
    if (key > root->data) return search(root->right, key);
    return 1;
}

int height(struct Node* root) {
    if (root == NULL) return -1;
    int left = height(root->left);
    int right = height(root->right);
    return (left > right ? left : right) + 1;
}

int depth(struct Node* root, int key) {
    int d = 0;
    while (root != NULL) {
        if (key < root->data) { root = root->left; d++; }
        else if (key > root->data) { root = root->right; d++; }
        else return d;
    }
    return -1;
}

int main() {
    struct Node* root = NULL;
    int choice, value;

    while (1) {
        printf("\n1 Insert\n2 Delete\n3 Search\n4 Height\n5 Depth\n6 Exit\nEnter choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("Enter value: ");
            scanf("%d", &value);
            root = insert(root, value);
        }
        else if (choice == 2) {
            printf("Enter value: ");
            scanf("%d", &value);
            root = deleteNode(root, value);
        }
```

```
        else if (choice == 3) {
            printf("Enter value: ");
            scanf("%d", &value);
            if (search(root, value)) printf("Found\n");
            else printf("Not Found\n");
        }
        else if (choice == 4) printf("Height = %d\n", height(root));
        else if (choice == 5) {
            printf("Enter value: ");
            scanf("%d", &value);
            printf("Depth = %d\n", depth(root, value));
        }
        else break;
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
1 Insert
2 Delete
3 Search
4 Height
5 Depth
6 Exit
Enter choice: 1
Enter value: 50

1 Insert
2 Delete
3 Search
4 Height
5 Depth
6 Exit
Enter choice: 1
Enter value: 20

1 Insert
2 Delete
3 Search
4 Height
5 Depth
6 Exit
Enter choice: 1
Enter value: 70

1 Insert
2 Delete
3 Search
4 Height
5 Depth
6 Exit
Enter choice: 3
Enter value: 20
Found

1 Insert
2 Delete
3 Search
4 Height
5 Depth
6 Exit
Enter choice: 4
Height = 1

1 Insert
2 Delete
3 Search
```

4 Height
5 Depth
6 Exit
Enter choice: 5
Enter value: 70
Depth = 1

II. Design an algorithm and a program to convert binary search tree created in previous question into balanced BST. Also find maximum and minimum element from this balanced BST.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 35

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* newNode(int x) {
   struct Node* t = (struct Node*)malloc(sizeof(struct Node));
   t->data = x;
   t->left = t->right = NULL;
   return t;
}

struct Node* insert(struct Node* root, int x) {
   if (root == NULL) return newNode(x);
   if (x < root->data) root->left = insert(root->left, x);
   else if (x > root->data) root->right = insert(root->right, x);
   return root;
}

int arr[100], n = 0;

void inorderStore(struct Node* root) {
   if (root == NULL) return;
   inorderStore(root->left);
   arr[n++] = root->data;
   inorderStore(root->right);
}

struct Node* buildBalanced(int s, int e) {
   if (s > e) return NULL;
   int mid = (s + e) / 2;
   struct Node* root = newNode(arr[mid]);
   root->left = buildBalanced(s, mid - 1);
   root->right = buildBalanced(mid + 1, e);
   return root;
}

struct Node* makeBalanced(struct Node* root) {
   n = 0;
   inorderStore(root);
   return buildBalanced(0, n - 1);
```

```
}

int findMin(struct Node* root) {
   while (root->left != NULL) root = root->left;
   return root->data;
}

int findMax(struct Node* root) {
   while (root->right != NULL) root = root->right;
   return root->data;
}

void inorderPrint(struct Node* root) {
   if (root == NULL) return;
   inorderPrint(root->left);
   printf("%d ", root->data);
   inorderPrint(root->right);
}

int main() {
   struct Node* root = NULL;
   int c, v;

   while (1) {
      printf("\n1 Insert\n2 Make Balanced\n3 Print Balanced\n4 Min\n5 Max\n6 Exit\nEnter choice: ");
      scanf("%d", &c);

      if (c == 1) {
         printf("Enter value: ");
         scanf("%d", &v);
         root = insert(root, v);
      }
      else if (c == 2) root = makeBalanced(root);
      else if (c == 3) { inorderPrint(root); printf("\n"); }
      else if (c == 4) printf("Min: %d\n", findMin(root));
      else if (c == 5) printf("Max: %d\n", findMax(root));
      else break;
   }

   return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
6 Exit
Enter choice: 1
Enter value: 70

1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
6 Exit
Enter choice: 1
Enter value: 30

1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
6 Exit
Enter choice: 1
Enter value: 40

1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
6 Exit
Enter choice: 2

1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
6 Exit
Enter choice: 3
20 30 40 50 70

1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
```

6 Exit
Enter choice: 4
Min: 20

1 Insert
2 Make Balanced
3 Print Balanced
4 Min
5 Max
6 Exit
Enter choice: 5
Max: 70

III.Given a binary search tree, design an algorithm and write a program to find which level number of tree has maximum number of nodes. Also print maximum number of nodes present at this level.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 36

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* newNode(int x) {
   struct Node* t = (struct Node*)malloc(sizeof(struct Node));
   t->data = x;
   t->left = t->right = NULL;
   return t;
}

struct Node* insert(struct Node* root, int x) {
   if (root == NULL) return newNode(x);
   if (x < root->data) root->left = insert(root->left, x);
   else if (x > root->data) root->right = insert(root->right, x);
   return root;
}

int maxNodesLevel(struct Node* root) {
   if (root == NULL) return -1;

   struct Node* q[100];
   int front = 0, rear = 0;
   q[rear++] = root;

   int level = 0, maxLevel = 0, maxNodes = 0;

   while (front < rear) {
      int count = rear - front;
      if (count > maxNodes) {
         maxNodes = count;
         maxLevel = level;
      }

      for (int i = 0; i < count; i++) {
         struct Node* temp = q[front++];
         if (temp->left) q[rear++] = temp->left;
         if (temp->right) q[rear++] = temp->right;
      }
```

```
        level++;
    }

    printf("Level with max nodes: %d\n", maxLevel);
    printf("Maximum nodes at this level: %d\n", maxNodes);

    return 0;
}

int main() {
    struct Node* root = NULL;
    int c, v;

    while (1) {
        printf("\n1 Insert\n2 Find Level with Max Nodes\n3 Exit\nEnter choice: ");
        scanf("%d", &c);

        if (c == 1) {
            printf("Enter value: ");
            scanf("%d", &v);
            root = insert(root, v);
        }
        else if (c == 2) maxNodesLevel(root);
        else break;
    }
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 1
Enter value: 23

1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 1
Enter value: 34

1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 1
Enter value: 35

1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 1
Enter value: 56

1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 1
Enter value: 65

1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 1
Enter value: 76

1 Insert
2 Find Level with Max Nodes
3 Exit
Enter choice: 2
Level with max nodes: 0
Maximum nodes at this level: 1
```

Week 13: I. Write an algorithm and a program to implement priority queue (also known as heap). Priority queue has following properties: a) Each item has a priority associated with it. b)Element which has heighest priority is dequeued before an element with low priority. c) If two elements have the same priority, they are served according to their order in the queue. d)It is always in the form of complete tree. e) Element which has highest priority should be at root, element which has priority less than root but more than other elements comes at level 2. Hence as the level increases, priority of element decreases. The program should implement following operations: a) create() - create root node of priority heap b)insert() - insert an element into the priority queue. Everytime when a node is inserted, tree should be balanced according to its priority. c) getHighestPriority() - finds the element which has highest priority d) deleteHeighestPriority() - delete the element which has highest priority.
NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 37

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int value;
   int priority;
};

struct Node heap[100];
int size = 0;

void swap(int i, int j) {
   struct Node temp = heap[i];
   heap[i] = heap[j];
   heap[j] = temp;
}

void insert(int val, int p) {
   size++;
   heap[size-1].value = val;
   heap[size-1].priority = p;

   int i = size - 1;
   while (i > 0 && heap[(i-1)/2].priority < heap[i].priority) {
      swap(i, (i-1)/2);
      i = (i-1)/2;
   }
}

struct Node getHighestPriority() {
   return heap[0];
}

void heapify(int i) {
   int largest = i;
```

```
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < size && heap[left].priority > heap[largest].priority)
        largest = left;

    if (right < size && heap[right].priority > heap[largest].priority)
        largest = right;

    if (largest != i) {
        swap(i, largest);
        heapify(largest);
    }
}

void deleteHighestPriority() {
    if (size == 0) return;

    heap[0] = heap[size - 1];
    size--;
    heapify(0);
}

int main() {
    int c, v, p;

    while (1) {
        printf("\n1 Insert\n2 Get Highest Priority\n3 Delete Highest Priority\n4 Exit\nEnter choice: ");
        scanf("%d", &c);

        if (c == 1) {
            printf("Enter value: ");
            scanf("%d", &v);
            printf("Enter priority: ");
            scanf("%d", &p);
            insert(v, p);
        }
        else if (c == 2) {
            if (size == 0) printf("Queue empty\n");
            else {
                struct Node t = getHighestPriority();
                printf("Value: %d Priority: %d\n", t.value, t.priority);
            }
        }
        else if (c == 3) {
            deleteHighestPriority();
            printf("Deleted highest priority element\n");
        }
        else break;
    }
    return 0;
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
1 Insert
2 Get Highest Priority
3 Delete Highest Priority
4 Exit
Enter choice: 1
Enter value: 50
Enter priority: 2

1 Insert
2 Get Highest Priority
3 Delete Highest Priority
4 Exit
Enter choice: 1
Enter value: 90
Enter priority: 5

1 Insert
2 Get Highest Priority
3 Delete Highest Priority
4 Exit
Enter choice: 2
Value: 90 Priority: 5

1 Insert
2 Get Highest Priority
3 Delete Highest Priority
4 Exit
Enter choice: 3
Deleted highest priority element

1 Insert
2 Get Highest Priority
3 Delete Highest Priority
4 Exit
Enter choice: 2
Value: 50 Priority: 2
```

II. Given a binary tree, design an algorithm and write a program to check whether this tree is heap or not. Tree should satisfy following heap properties: a) Tree should be complete binary tree b)Every nodes value should be greater than or equal to its child node (incase of max heap)

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 38

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* newNode(int val) {
   struct Node* t = (struct Node*)malloc(sizeof(struct Node));
   t->data = val;
   t->left = t->right = NULL;
   return t;
}

int countNodes(struct Node* root) {
   if (!root) return 0;
   return 1 + countNodes(root->left) + countNodes(root->right);
}

int isComplete(struct Node* root, int index, int total) {
   if (!root) return 1;
   if (index >= total) return 0;
   return isComplete(root->left, 2*index+1, total) &&
         isComplete(root->right, 2*index+2, total);
}

int isMaxHeap(struct Node* root) {
   if (!root->left && !root->right) return 1;
   if (!root->right)
      return (root->data >= root->left->data) && isMaxHeap(root->left);
   return (root->data >= root->left->data && root->data >= root->right->data) &&
         isMaxHeap(root->left) && isMaxHeap(root->right);
}

int isHeap(struct Node* root) {
   int total = countNodes(root);
   return isComplete(root, 0, total) && isMaxHeap(root);
}

int main() {
   struct Node* root = newNode(50);
```

```
    root->left = newNode(40);
    root->right = newNode(30);
    root->left->left = newNode(35);
    root->left->right = newNode(20);

    if (isHeap(root))
        printf("Tree is a Max Heap\n");
    else
        printf("Tree is NOT a Max Heap\n");

    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
        50
       /  \
     40    30
    /  \
   35  20
```

Tree is a Max Heap

III.Given a complete binary tree, design an algorithm and write a program to find Kth largest element in it. (Hint: Max heap)

NAME - KRITIKA DHIMAN
SECTION - DS1
ROLL NO - 31
QUESTION NO - 39

```
#include <stdio.h>

void maxHeapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        maxHeapify(arr, n, largest);
    }
}void buildMaxHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        maxHeapify(arr, n, i);}

int extractMax(int arr[], int *n) {
    int max = arr[0];
    arr[0] = arr[*n - 1];
    (*n)--;

    maxHeapify(arr, *n, 0);
    return max;
}int main() {
    int n, k;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &k);

    buildMaxHeap(arr, n);

    int x;
    for (int i = 0; i < k; i++)
```

```
        x = extractMax(arr, &n);

    printf("%d\n", x);
    return 0;
}
```

```
PS C:\Users\Desktop\KRITIKA31> cd "c:\Users\Desktop\KRITIKA31 \" ; if ($?) { gcc que11.c -o que11
} ; if ($?) { .\que11 }
Enter number of nodes: 6
Enter tree elements: 20 30 40 50 60 40
Enter value of K: 3
Kth largest element = 40
```